IES Joan Coromines
Benicarló

Unidad didáctica

XML



El lenguaje XML

El nombre **XML** viene de *extensible markup language*. El origen de XML está ligado al de HTML: mientras que HTML sería como un texto con anotaciones que indican el formato o la presentación, XML surgió como un lenguaje similar a HTML pero para representar exclusivamente datos de cualquier tipo.

Entre los principios de diseño de XML está el que sea **fácil de leer e interpretar** tanto por humanos como por máquinas. Esto se ha conseguido a cambio de una gran verbosidad.

Los documentos XML ocupan mucho espacio en relación con la cantidad efectiva de información de contienen.

Estructura de un documento XML

Un documento XML es un documento de texto escrito conforme a la sintaxis del lenguaje XML. Se identifican principalmente tres componentes:

1. **Cabecera**: se identifica el documento como XML y se proporcionan detalles como la versión y la codificación empleada.
2. **Elementos**: todo el documento está comprendido entre una etiqueta de apertura y una etiqueta de cierre. Un bloque de este tipo es un elemento.
3. **Atributos**: aparecen en las etiquetas de apertura de los elementos y consisten en la asignación de un valor a un nombre.

```
<?xml version="1.0" encoding="UTF-8" ?>
<clientes>
        <cliente DNI="123456789A">
                <apellidos>NADALES</apellidos>
                <CP>44126</CP>
        </cliente>
        <cliente DNI="456781459B">
                <apellidos>SAMPER</apellidos>
                <CP>12500</CP>
        </cliente>
</clientes>
```

Persistencia de datos usando JAXB

Una de las formas más habituales de **almacenar información** es mediante una base de datos. Las bases de datos típicamente contienen algún tipo de datos relacionales que puede llegar a ser costoso de adaptar.

Para un sencillo modelo de datos es mucho más fácil usar **XML**. Para ello se utiliza una librería llamada **JAXB** (*Java Architecture for XML Binding*). Con unas pocas líneas de código, JAXB nos permitirá generar una salida XML como esta:

```
<persons>
  <person>
    <birthday>1999-02-21</birthday>
    <city>some city</city>
    <firstName>Hans</firstName>
    <lastName>Muster</lastName>
    <postalCode>1234</postalCode>
    <street>some street</street>
  </person>
  <person>
    <birthday>1999-02-21</birthday>
    <city>some city</city>
    <firstName>Anna</firstName>
    <lastName>Best</lastName>
    <postalCode>1234</postalCode>
    <street>some street</street>
  </person>
</persons>
```

Utilización de JAXB

JAXB proporciona dos funcionalidades principales: la capacidad de **convertir objetos Java en XML y viceversa**.

Para que JAXB sea capaz de hacer la conversión, es necesario preparar el modelo. Partiendo del ejemplo de XML anterior, se genera la clase Person.java.

```
public class Person {
    private String birthday;
    private String city;
    private String firstName;
    private String lastName;
    private String postalCode;
    private String street;

    public Person() { }

    public Person(String birthday, String city, String firstName,
String lastName, String postalCode, String street) {
        this.birthday = birthday;
        this.city = city;
        this.firstName = firstName;
        this.lastName = lastName;
        this.postalCode = postalCode;
        this.street = street;
    }

    // Getters y setters
}
```

Preparación del modelo para JAXB

Se necesita una clase adicional para contener la lista de personas de cara a ser adaptada a XML por JAXB. Esta clase se llamará PersonListWrapper.

```
import java.util.List;

import javax.xml.bind.annotation.XmlElement;
import javax.xml.bind.annotation.XmlRootElement;

@XmlRootElement(name = "persons")
public class PersonListWrapper {

    private List<Person> persons;

    @XmlElement(name = "person")
    public List<Person> getPersons() {
        return persons;
    }

    public void setPersons(List<Person> persons) {
        this.persons = persons;
    }
}
```

Para poder usar las librerías de JAXB hay que añadir las siguientes dependencias en el archivo build.gradle:

```
dependencies {
    ...
    implementation 'javax.xml.bind:jaxb-api:2.3.1'
    implementation 'org.glassfish.jaxb:jaxb-runtime:2.3.1'
    implementation 'javax.activation:activation:1.1.1'
}
```

Leer y escribir datos con JAXB

Por último, se añaden los siguientes métodos para leer y escribir los datos XML en la clase Main correspondiente.

```
// Lee los datos de un archivo XML y lo convierte en una lista de personas
public static ArrayList<Person> loadPersonDataFromFile(File file) {
    try {
        JAXBContext context =
JAXBContext.newInstance(PersonListWrapper.class);
        Unmarshaller um = context.createUnmarshaller();

        // Lee los datos desde el archivo XML y lo deserializa
        PersonListWrapper wrapper = (PersonListWrapper) um.unmarshal(file);

        // Convierte los datos en una lista de objetos
        ArrayList<Person> personData = new ArrayList<>();
        personData.addAll(wrapper.getPersons());
        return personData;
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

```
// Guarda los datos de la lista de personas en un archivo XML
public static void savePersonDataToFile(File file, ArrayList<Person> personData) {
    try {
        JAXBContext context =
JAXBContext.newInstance(PersonListWrapper.class);
        Marshaller m = context.createMarshaller();
        m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);

        // Guarda los datos en el archivo XML
        PersonListWrapper wrapper = new PersonListWrapper();
        wrapper.setPersons(personData);

        // Escribe los datos de la lista sobre el archivo XML
        m.marshal(wrapper, file);
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

Mostrar datos leídos

Un ejemplo para comprobar que se están leyendo correctamente los datos del archivo XML ubicado en la carpeta de *resources* sería mediante una llamada al método anterior:

```
public static void main(String[] args) {
    try {
        // Carga los datos desde el archivo personas.xml (ubicado en la carpeta “res”)
        File archivoXML = new File(App.class.getResource("/personas.xml").toURI())
        ArrayList<Person> personData = loadPersonDataFromFile(archivoXML);

        // Muestra por pantalla los datos cargados en la lista
        personData.forEach(person -> System.out.println(person.getFirstName() + " " + person.getLastName() + ", "
            + person.getBirthday() + ", " + person.getStreet() + ", " + person.getCity() + ", " + person.getPostalCode()));

    } catch (URISyntaxException e) {
        e.printStackTrace();
    }
}
```

Se obtienen los objetos Persona en una la lista y se muestran sus datos por pantalla.

```
Hans Muster, 1999-02-21, some street, some city, 1234
Anna Best, 1999-02-21, some street, some city, 1234
```

Ejercicio entregable #12

Crea un archivo XML llamado biblioteca.xml que contenga información sobre, al menos, cinco libros. Cada libro deberá tener los siguientes datos: título, autor, año de publicación y precio.

A continuación, escribe un programa en Java que lea dicho archivo y muestre su contenido por la pantalla.

Lectura de atributos con JAXB

Siguiendo el ejemplo anterior se ha añadido un atributo id para cada elemento <person>.

```
<persons>
    <person id="1">
        <birthday>1999-02-21</birthday>
        <city>some city</city>
```

Para poder leer dicho atributo en el modelo, se añade la anotación @XmlAttribute en la clase Person encima de su método *get*:

```
public class Person {
   private Integer id;
   private String birthday;
   private String city;
   ...

   @XmlAttribute(name = "id")
   public Integer getId() {
      return id;
   }
   ...
}
```

Lectura de atributos con JAXB

En el método que muestra los datos por pantalla sólo será necesario añadir la llamada al método *get* del atributo en el bucle que itera la lista de personas.

```
public static void main(String[] args) {
   try {
      // Carga los datos desde el archivo personas.xml (ubicado en la carpeta “res”)
      File archivoXML = new File(App.class.getResource("/personas.xml").toURI())
      ArrayList<Person> personData = loadPersonDataFromFile(archivoXML);

      // Muestra por pantalla los datos cargados en la lista
      personData.forEach(person -> System.out.println(“id=” + person.getId() +  person.getFirstName() + " " + person.getLastName() + ", "
+ person.getBirthday() + ", " + person.getStreet() + ", " + person.getCity() + ", " + person.getPostalCode()));

   } catch (URISyntaxException e) {
      e.printStackTrace();
   }
}
```

```
id=1Hans Muster, 1999-02-21, some street, some city, 1234
id=2Anna Best, 1999-02-21, some street, some city, 1234
```

Ejercicio entregable #13

Crea un archivo XML llamado peliculas.xml que contenga información sobre, al menos, cinco películas. Cada película deberá tener los siguientes elementos: título, director, año, género y duración (en minutos), además de un atributo *clasificación* con la edad mínima recomendada para dicha película.

A continuación, escribe un programa en Java que lea dicho archivo y muestre por pantalla aquellas películas con una clasificación mayor a 13.

Añadir nuevos datos al modelo

Sobre el modelo de datos de personas anterior, se quiere guardar una lista con contactos que tiene cada persona. Por ejemplo:

```
<persons>
  <person>
    <birthday>1999-02-21</birthday>
    <city>some city</city>
    <firstName>Hans</firstName>
    <lastName>Muster</lastName>
    <postalCode>1234</postalCode>
    <street>some street</street>
    <contacts>
      <contact>Elena</contact>
      <contact>Laura</contact>
      <contact>Juan</contact>
    </contacts>
  </person>
  ...
</persons>
```

Añadir nuevos datos al modelo

En la clase Person.java del modelo habrá que añadir una lista de *strings* para representar los nuevos datos sobre los contactos, así como su correspondiente inicialización en el constructor y sus métodos *get* y *set*.

JAXB hará la serialización automáticamente de cada elemento *contact* encontrado en el XML dentro de dicha lista.

```
public class Person {
    private String birthday;
    private String city;
    private String firstName;
    private String lastName;
    private String postalCode;
    private String street;
    private List<String> contacts;

    public Person() { }

    public Person(String birthday, String city, String firstName, String lastName, String
postalCode, String street, List<String> contact) {
        // Resto de inicializaciones
        this.contacts = contacts;
    }

    // Getters y setters
    @XmlElementWrapper(name = "contacts")
    @XmlElement(name = "contact")
    public List<String> getContacts() {
        return contacts;
    }

    public void setContacts(List<String> contacts) {
        this.contacts = contacts;
    }
}
```

Añadir nuevos datos al modelo

Una vez realizados estos cambios sobre el modelo, queda mostrar los nuevos datos por la terminal:

```
public static void main(String[] args) {
      // Carga los datos desde el archivo personas.xml (ubicado en la carpeta “res”)
      File archivoXML = new File(App.class.getResource("/personas.xml").toURI())
      ArrayList<Person> personData = loadPersonDataFromFile(archivoXML);

      // Muestra por pantalla los datos cargados en la lista
     for (int i = 0; i < personData.size(); i++) {
           Person p = personData.get(i);
           System.out.println("Person " + i + ": id=" + p.getId() + p.getFirstName() + " " + p.getLastName() + ", " + p.getBirthday() + ", " +
p.getStreet() + ", " + p.getCity() + ", " + p.getPostalCode());

           // Para cada persona, muestra sus contactos
           for (int j = 0; j < p.getContacts().size(); j++) {
              System.out.println("   Contact: " + p.getContacts().get(j));
           }
     }
   } catch (URISyntaxException e) {
      e.printStackTrace();
   }
}
```

```
Person 0: id=1Hans Muster, 1999-02-21, some street, some city, 1234
   Contact: Elena
   Contact: Laura
   Contact: Juan
Person 1: id=2Anna Best, 1999-02-21, some street, some city, 1234
   Contact: Sandra
   Contact: Paul
   Contact: Rosa
```

Ejercicio entregable #14

Diseña una aplicación en Java que gestione un archivo XML llamado videojuegos.xml utilizando JAXB. El archivo contendrá varios videojuegos con los datos título, año de lanzamiento, desarrollador, género y una lista de plataformas en las que está disponible. La aplicación deberá permitir:

1. Mostrar todos los videojuegos almacenados.
2. Insertar un nuevo videojuego.
3. Eliminar un elemento por su título.

Añadir nuevos datos al modelo

Siguiendo con el modelo de datos de personas, se quiere añadir ahora una lista de teléfonos para cada uno de los contactos de cada persona.

El nombre de cada contacto se escribe ahora dentro de la nueva etiqueta de <name>. Para la lista de teléfonos, se utiliza la etiqueta <phones> que contendrá elementos <phone>.

```
<persons>
  <person>
    <birthday>1999-02-21</birthday>
    <city>some city</city>
    <firstName>Hans</firstName>
    <lastName>Muster</lastName>
    <postalCode>1234</postalCode>
    <street>some street</street>
    <contacts>
        <contact>
                <name>Elena</name>
                <phones>
                  <phone>123456789</phone>
                  <phone>987654321</phone>
                </phones>
        </contact>
        <contact>
                <name>Laura</name>
                <phones>
                  <phone>111111111</phone>
                </phones>
        </contact>
      ...
    </contacts>
  </person>
  ...
</persons>
```

Añadir nuevos datos al modelo

En la clase Person.java del modelo habrá que cambiar la lista de strings por una nueva clase llamada **Contact.java**, que representará los datos de cada contacto en el XML.

JAXB utilizará la nueva clase para hacer la serialización automáticamente de cada elemento encontrado en el XML dentro de dicha lista.

```
public class Person {
    private String birthday;
    private String city;
    private String firstName;
    private String lastName;
    private String postalCode;
    private String street;
    private List<Contact> contacts;

    public Person() { }

    public Person(String birthday, String city, String firstName, String lastName, String
postalCode, String street, List<Contact> contact) {
        // Resto de inicializaciones
        this.contacts = contacts;
    }

    // Getters y setters
    @XmlElementWrapper(name = "contacts")
    @XmlElement(name = "contact")
    public List<Contact> getContacts() {
        return contacts;
    }

    public void setContacts(List<Contact> contacts) {
        this.contacts = contacts;
    }
}
```

Añadir nuevos datos al modelo

A continuación, se crea la clase Contact.java para representar los nuevos datos del modelo reflejado en el archivo XML.

```java
public class Contact {
    private String name;
    private List<String> phones;

    public Contact() {}

    public Contact(String name, List<String> phones) {
        this.name = name;
        this.phones = phones;
    }

    @XmlElement(name = "name")
    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    @XmlElementWrapper(name = "phones")
    @XmlElement(name = "phone")
    public List<String> getPhones() {
        return phones;
    }

    public void setPhones(List<String> phones) {
        this.phones = phones;
    }
}
```

Añadir nuevos datos al modelo

Una vez realizados estos cambios sobre el modelo, queda mostrar los nuevos datos por la terminal:

```
public static void main(String[] args) {
      // Carga los datos desde el archivo personas.xml (ubicado en la carpeta “res”)
      File archivoXML = new File(App.class.getResource("/personas.xml").toURI())
      ArrayList<Person> personData = loadPersonDataFromFile(archivoXML);
      // Muestra por pantalla los datos cargados en la lista
     for (int i = 0; i < personData.size(); i++) {
           Person p = personData.get(i);
           System.out.println("Person " + i + ": id=" + p.getId() + p.getFirstName() + " " + p.getLastName() + ", "
                 + p.getBirthday() + ", " + p.getStreet() + ", " + p.getCity() + ", " + p.getPostalCode());

           // Para cada persona, muestra sus contactos
           for (int j = 0; j < p.getContacts().size(); j++) {
                 Contact c = p.getContacts().get(j);
                 System.out.println("   Contacto: " + c.getName());
                 // Lista de teléfonos del contacto
                 for (int k = 0; k < c.getPhones().size(); k++) {
                       System.out.println("      Teléfono: " + c.getPhones().get(k));
                 }
           }
     }
   } catch (URISyntaxException e) {
     e.printStackTrace();
   }
}
```

Ejercicio entregable #15

Se dispone de un archivo XML llamado centros.xml que contiene información sobre varios centros educativos. Cada centro incluye los siguientes datos:

- Atributo id.
- Nombre, ciudad, dirección y código postal.
- Una lista de profesores, donde cada profesor contiene:
  - Nombre
  - Departamento
  - Una lista de asignaturas que imparte.

Crea un programa en Java que realice las siguientes tareas:

- Lea el archivo XML y muestre todos los datos por la consola.
- Cuente y muestre el total de profesores del departamento de Informática.